Connecting the model to the money

QS Cost
July 30, 2026

CONTENTS

Connecting the model to the money

How school_str.ifc was bound to Tower_X_Project_Data.xlsx

1. The problem: the two files share no key

No source data was modified

2. Three mechanisms, not one

3. Mechanism A — the authored element register

3.1 Why one declared hop buys the whole chain

3.2 Generating the register

3.3 The register file

3.4 The mapping rules in full

3.5 Loading the register

3.6 Where the model actually meets the workbook

3.7 Resolving it against the loaded geometry

4. Mechanism B — take-off pricing

4.1 Measuring a model with no quantities

4.2 Four declared pricing rules

4.3 The unit guard — which is not a converter

4.4 Reconciliation and comparison

5. Mechanism C — zone map and cost link

6. The 4D build sequence

7. Every assumption

Assumption 1 — The model is not the building the bill is for

Assumption 2 — Rebar is placed by storey, because the file carries no host relationship

Assumption 3 — Formwork rows are inferred, not measured

Assumption 4 — 399 elements are deliberately left unmapped

Assumption 5 — An element’s confidence is its weakest binding

Assumption 6 — An element’s alert is its worst cost centre

Assumption 7 — 9_HISTORICAL_DATA.Discipline is truncated in the source data

Assumption 8 — Which elements are built, and in what order

Assumption 9 — There is no unit conversion anywhere in the pipeline

Assumption 10 — Unit rates exclude margin and contingency

Assumption 11 — Sheet geometry is hardcoded

Assumption 12 — AC_AED_Period is cumulative despite its name

Assumption 13 — A zone’s CPI is not painted below 1% of budget spent

Assumption 14 — The register is not regenerated by any build step

8. Where the three mechanisms disagree

9. Verification and regeneration

Connecting the model to the money

HOW `SCHOOL_STR.IFC` WAS BOUND TO `TOWER_X_PROJECT_DATA.XLSX`

Status: built · **Date:** 2026-07-30 **Subjects:** `QsEarlyWarning/frontend/qs-early-warning/public/models/school_str.ifc` · `data/Tower_X_Project_Data.xlsx` · `data/ifc_boq_map.csv`

---

1. The problem: the two files share no key

`school_str.ifc` is a genuine Autodesk Revit 2024 structural export — 8.2 MB, IFC4, 1,526 considered elements across five storeys. **Not one of those elements carries a cost code.** There is no `IfcElementQuantity` anywhere in the file, no cost property set, no BOQ reference. The information is simply not there, and no amount of parsing will find it.

The other direction is equally empty. The workbook's `1_BOQ` carries `Item Ref`, `Norm Ref`, descriptions, quantities and rates — and no `IfcGlobalId`, no element identifier of any kind.

So there is nothing to join on. Every link between geometry and money in this system is therefore either **authored** (someone declared it) or **derived by rule** (a class-and-storey heuristic). None of it is discovered. The whole credibility of the 3D and 4D features rests on those choices being visible, reviewable and arguable — which is what this document is for.

NO SOURCE DATA WAS MODIFIED

File	Size	MD5	Status
`data/Tower_X_Project_Data.xlsx`	633,199 B	`daabf42984e-f59e909a5f46c5ee44cb4`	**unmodified**
`.../public/models/school_str.ifc`	8,593,426 B	`d7c500de611b85fea19c-cfd92996cba3`	**unmodified**
`data/ifc_boq_map.csv`	322,902 B	`296cee259dc-c95370d976d92ccae70f7`	new sidecar

`CLAUDE.md` marks the workbook read-only source. Adding a sheet to it was considered and rejected: it would fork the file we were handed from the one the importer's contract tests are pinned to (`DataContractTests` asserts 2,076 rows / 173 cost centres). A CSV beside it stays diffable in git, opens in the same spreadsheet, and can be hand-edited without touching source data.

---

2. Three mechanisms, not one

The model reaches the sheet by three separate routes. They use different rules, different grains and — a real trap — three different confidence scales that look alike. Conflating them is the easiest mistake to make when reading this codebase, so they are separated here first.

	Mechanism	Grain	Artefact	Drives
A	Authored element register	element → BOQ item → cost centre	`data/ifc_boq_map.csv`	click-through, per-element cost, the 4D build sequence
B	Take-off pricing	IFC class + measure → BOQ rate	`TakeoffRateMap.cs`	priced take-off, quantity variance against the bill
C	Zone map + cost link	class + storey → `Zone_Area`	`ifcZoneMap.ts`, `ifcCostLink.ts`	fallback painting, the match-rate honesty metric

Mechanism A is the main event and the only one that reaches earned value. B answers a different question ("does the model agree with the bill?"). C is the fallback for any model the register does not cover, and exists mainly to report *how much it cannot place*.

---

3. Mechanism A — the authored element register

3.1 WHY ONE DECLARED HOP BUYS THE WHOLE CHAIN

The reason it was worth authoring this cheaply rather than elaborately:

```
IFC element  (GlobalId)
    │
    │  AUTHORED — the only judgement in the chain
    │  data/ifc_boq_map.csv
    ▼
BOQ Item Ref  (1_BOQ, e.g. 2.04)
    │
    │  REAL — 9_HISTORICAL_DATA.WBS_Code IS the BOQ Item Ref
    │  173 vs 173 · intersection 173 · zero orphans
    ▼
BCC_ID  (cost centre)
    │
    ▼
12 periods of BAC / PV / EV / AC / CPI / SPI / alert
```

That second hop is **not authored**. It is an exact, bijective join already present in the supplied workbook — every one of the 173 `WBS_Code` values in `9_HISTORICAL_DATA` is a `1_BOQ` `Item Ref`, and every `Item Ref` is a `WBS_Code`, with no orphans on either side. It is verified in `IfcElementMapTests.The_boq_item_ref_and_the_wbs_code_are_the_same_key`.

One declared arrow therefore buys everything downstream: a click on a column reaches its bill items, their rates, their cost centres, and those centres' twelve periods of earned value. **Only the first arrow is a judgement.** Everything after it is source data.

3.2 GENERATING THE REGISTER

`tools/ifc_boq_map/generate_map.py` reads the STEP file with **plain regex — no ifcopenshell**. Only four fields are needed, and a build-time native dependency to read four fields would not pay for itself:

```
# #123=IFCCOLUMN('guid',#4,'Name',...)
# GlobalId is always the first argument.
r"#(\d+)\s*=\s*(IFC[A-Z0-9]+)\s*\(\s*'([^']*)'"

# #9=IFCBUILDINGSTOREY('guid',#4,'01 - Entry Level',...)
# Name is the third argument.
r"#(\d+)\s*=\s*IFCBUILDINGSTOREY\s*\(\s*'[^']*'\s*,\s*[^,]*,\s*'([^']*)'"

# IFCRELCONTAINEDINSPATIALSTRUCTURE(
#     guid, owner, name, desc, (elements...), storey)
r"IFCRELCONTAINEDINSPATIALSTRUCTURE\s*\((.*?)\)\s*;"
```

Fifteen classes are considered — deliberately the same list as `MEASURED_CLASSES` in `ifcMeasure.ts`, so the coverage percentages the UI reports share a denominator with the take-off:

```
CONSIDERED = [
    "IFCSLAB", "IFCCOLUMN", "IFCBEAM", "IFCWALL",
    "IFCWALLSTANDARDCASE", "IFCMEMBER", "IFCPLATE", "IFCFOOTING",
    "IFCPILE", "IFCCOVERING", "IFCCURTAINWALL", "IFCREINFORCINGBAR",
    "IFCSTAIR", "IFCRAMP", "IFCROOF",
]

DIRECT = 0.9      # the class itself is the evidence
INFERRED = 0.6    # the storey stands in for a missing relationship
```

The entire matching logic is one function. It is **pure class lookup plus a single storey test** — there is no keyword matching, no fuzzy matching, no name parsing anywhere in the chain:

```
def rules(ifc_class, storey):
    if ifc_class == "IFCCOLUMN":
        return [("2", "2.04", "Concrete", "Column concrete...", DIRECT),
                ("2", "2.09", "Formwork", "...implied by the column", DIRECT)]
    if ifc_class in ("IFCWALL", "IFCWALLSTANDARDCASE"):
        return [("2", "2.05", "Concrete", ..., DIRECT),
                ("2", "2.10", "Formwork", ..., DIRECT)]
    if ifc_class == "IFCSLAB":
        return [("2", "2.06", "Concrete", ..., DIRECT),
                ("2", "2.11", "Formwork", ..., DIRECT)]
    if ifc_class == "IFCREINFORCINGBAR":
        if storey == SUB_LEVEL:                 # the only storey test
            return [("2", "2.12", "Rebar", "...raft...", INFERRED)]
        return [("2", "2.14", "Rebar", "...suspended slab...", INFERRED)]
    return []                # ← everything else is the scope gap
```

Output is deterministic — elements are sorted by `(GlobalId, class)` so dictionary ordering cannot leak into the file. Run it twice and the diff is empty.

3.3 THE REGISTER FILE

`data/ifc_boq_map.csv` — 2,034 rows plus header, eight columns, **one row per (GlobalId, BOQ item)** because a physical element consumes more than one bill item:

```
IfcGlobalId,IfcClass,Storey,BoqSec,BoqItemRef,Role,Basis,Confidence
01SfNHv5nEReC9M9Bzo7D_,IFCWALL,01 – Entry Level,2,2.05,Concrete,
  "Structural wall concrete, priced per m3 of wall.",0.9
```

The `Basis` column is the point of the file: every row carries, in words, why that binding is defensible, and it ships verbatim to the UI so a QS can disagree with it.

Elements the bill cannot price are written into the file too, with an empty `BoqItemRef`, `Role="Unmapped"`, `Confidence=0.0` and a reason. The scope gap travels with the mapping rather than having to be recomputed — an element the bill cannot price is a finding, not an absence.

Verified distributions (recomputed from the committed CSV for this document):

IfcClass	rows	elements	BOQ items
IFCREINFORCINGBAR	619	619	560 × 2.12 · 59 × 2.14
IFCSLAB	598	299	2.06 + 2.11
IFCCOLUMN	406	203	2.04 + 2.09
IFCBEAM	375	375	*none — scope gap*
IFCMEMBER	22	22	*none — scope gap*
IFCWALL	12	6	2.05 + 2.10
IFCPLATE	2	2	*none — scope gap*

Storeys, by row: `01 – Entry Level` 817 · `Sub Level` 560 · `02 – Floor` 244 · `03 – Floor` 235 · `Roof` 109 · none 69. Confidence, by row: `0.9` 1,016 · `0.6` 619 · `0.0` 399.

3.4 THE MAPPING RULES IN FULL

IFC class	Storey	BOQ item	Role	Elements	Confidence
IFCCOLUMN	any	2.04 Columns concrete C40/50	Concrete	203	0.9
IFCCOLUMN	any	2.09 Column formwork	Formwork	203	0.9
IFCWALL	any	2.05 Structural wall concrete	Concrete	6	0.9
IFCWALL	any	2.10 Wall formwork	Formwork	6	0.9
IFCSLAB	any	2.06 Suspended slab concrete	Concrete	299	0.9
IFCSLAB	any	2.11 Slab soffit formwork	Formwork	299	0.9
IFCREINFORCINGBAR	`Sub Level`	2.12 Rebar — raft foundations	Rebar	560	**0.6**
IFCREINFORCINGBAR	above ground	2.14 Rebar — suspended slabs	Rebar	59	**0.6**

Coverage: 1,127 of 1,526 elements (74%), 1,635 mapped rows, reaching **8 cost centres** — `BCC-STR-CON-204/205/206`, `BCC-STR-FWK-209/210/211`, `BCC-STR-RBR-212/214`.

3.5 LOADING THE REGISTER

`QsEarlyWarning.Infrastructure/Excel/IfcElementMapCsvLoader.cs`. Three things in it are worth knowing:

Rows fold onto elements, and confidence takes the minimum.

```
// An element is only as well-placed as its shakiest binding.
elementMeta[guid] = (meta.Cls, meta.Storey, Math.Min(meta.Conf, conf));
```

The CSV is parsed properly, not split on commas. `SplitCsv` is a minimal RFC-4180 reader, because the `Basis` column contains commas inside quotes — a naive `Split(',')` would shear every rationale in the file, and the test suite pins that it does not.

The register is project-gated. `TryLoadForProject` returns `null` unless the requested project is the one the register was authored for (`Data:EstimateProjectSlug`, default `tower-x`). Any other project gets the feature **absent rather than wrong**, and load failures degrade the same way.

3.6 WHERE THE MODEL ACTUALLY MEETS THE WORKBOOK

`ModelController.ElementMap` — `GET /api/v1/model/element-map`. The entire join is six lines:

```
// WBS_Code IS the BOQ item ref — an exact 1:1 in the source data.
// Reading it off the panel rather than re-deriving it keeps this
// endpoint honest about where the link comes from.
var bccByItemRef = snapshot.Panel
    .Where(p => !string.IsNullOrWhiteSpace(p.WbsCode))
    .GroupBy(p => p.WbsCode!.Trim(), StringComparer.OrdinalIgnoreCase)
    .ToDictionary(g => g.Key, g => g.First().BccId,
                  StringComparer.OrdinalIgnoreCase);

var items = map.BoqItemRefs.Select(itemRef => {
    var rate = rates?.Find(itemRef);                  // ← 1_BOQ, via RateBook
    return new MappedItemDto(itemRef, rate?.Description, rate?.Unit,
        rate?.UnitRate ?? 0, rate?.BoqQuantity,
        // ← 9_HISTORICAL_DATA
        bccByItemRef.TryGetValue(itemRef, out var bcc) ? bcc : null);
}).ToList();
```

Matching is case-insensitive and trimmed on both sides. The response carries its own provenance statement, which the UI shows rather than paraphrases:

> Element-to-BOQ bindings are authored and shipped in `data/ifc_boq_map.csv`; a real IFC export carries no cost codes, so this hop cannot be discovered. Everything downstream is source data: the BOQ item ref is the cost centre's `WBS_Code`.

Unit rates come from `RateBook.cs`, which divides `Direct+Indirect Amount` by `Quantity` and drops any line without a positive rate. **Margin and contingency are excluded** — this is a cost take-off, not a price.

Note that `BoqSec` is written by the generator and **never read** by the loader or the API. BOQ *sections* play no part in the join; the connection is item-ref-only.

3.7 RESOLVING IT AGAINST THE LOADED GEOMETRY

`ifcElementMap.ts` performs the last hop — the register's `IfcGlobalId` to the local integer id the Fragments model paints and picks by — using the model's own index:

```
const localIds = await model.getLocalIdsByGuids(
  map.elements.map((e) => e.globalId));

// register/model drift, counted
if (typeof localId !== "number") { notInModel++; continue; }
```

`notInModel` is the drift detector: it counts register rows the loaded file does not contain. It is **0** for the bundled pair and is surfaced in the UI if the two ever diverge.

Two display rules live here. Confidence is banded, not printed as a number:

```
if (confidence >= 0.9) return "Declared by element class";
if (confidence > 0)   return "Inferred from storey";
return "No bill item";
```

And an element bound to several cost centres takes the **worst** alert among them, never the average — an unknown alert level sorts between GREEN and AMBER rather than being dropped.

---

4. Mechanism B — take-off pricing

A separate, narrower route that answers "does this model agree with the bill?" rather than "what is this element costing".

4.1 MEASURING A MODEL WITH NO QUANTITIES

`ifcMeasure.ts` cannot use IFC BaseQuantities, because the sample model has none. It reads property sets instead, through a **multi-locale synonym table** — the Revit export's parameter group is in Spanish:

```
const VOLUME_KEYS = ["volumen", "volume", "netvolume", "grossvolume",
  "net volume", "gross volume", "volumen neto", "volumen bruto"];

const AREA_KEYS = ["area", "área", "netarea", "grossarea", "net area",
  "gross area", "netsidearea", "área neta", "area neta"];

const STANDARD_KEYS = ["netvolume", "grossvolume", "netarea",
  "grossarea", "netsidearea"];
```

Matching is **exact on the lower-cased property name**, and the first synonym in array order wins — so `volumen` beats `volume`. If a model yields quantities but none of `STANDARD_KEYS` was seen, the result reports `baseQuantitiesEmpty`, so the UI can say the take-off rode on exporter parameters rather than the standard. A model with no quantities and no recognisable parameter names reports itself unmeasurable rather than guessing.

Class queries are anchored — `new RegExp('^IFCWALL$', 'i')` — so `IFCWALL` does not sweep in `IFCWALLSTANDARDCASE`. Storeys come from `IfcRelContainedInSpatialStructure`; 69 elements in this model sit in no storey and are carried as `"(none)"`.

4.2 FOUR DECLARED PRICING RULES

`TakeoffRateMap.cs` — note this is a **different and smaller set** than the register's eight items:

IFC class	Measure	Unit	BOQ item
IFCCOLUMN	volume	m³	2.04 columns concrete C40/50
IFCWALL	volume	m³	2.05 structural wall concrete
IFCSLAB	volume	m³	2.06 suspended slab concrete
IFCSLAB	area	m²	2.11 slab soffit formwork

The gaps are deliberate and documented in words a QS can act on. `IFCBEAM`: *"this rate library prices no beam concrete — the BOQ has no beam item. Pricing it at the slab rate would invent a number the estimate never contained."* `IFCREINFORCINGBAR`: *"this library prices rebar by the tonne; converting bar geometry to tonnage needs a steel density and a bar schedule this model does not carry."*

4.3 THE UNIT GUARD — WHICH IS NOT A CONVERTER

`TakeoffPricer.cs`:

```
private static bool UnitsAgree(string ruleUnit, string? boqUnit) =>
    Normalise(ruleUnit) == Normalise(boqUnit);

private static string Normalise(string? unit) =>
    (unit ?? "").Trim().ToLowerInvariant()
        .Replace("³", "3").Replace("²", "2")
        .Replace("cum", "m3").Replace("sqm", "m2").Replace(" ", "");
```

This is the **only unit logic in the entire system**. It guarantees a volume can never be priced at an area rate — a mismatch lands the line in the unpriced residual with the reason stated. It does **not** convert anything (see assumption 9).

4.4 RECONCILIATION AND COMPARISON

The tie-out is `priced + unpriced + unmeasured == totalElements`, with the element count derived **independently of the pricing pass** so a pricing bug cannot hide behind a matching total.

Comparison to the bill is grouped by BOQ item, not by IFC class:

```
// positive = the model carries more than was priced
double variance = modelQuantity − boqQuantity;
// VariancePct = variance / boqQuantity
// CostImpact  = Math.Round(variance ∗ rate.UnitRate, 2)
```

An item whose BOQ quantity is missing or ≤ 0 is reported as **uncomparable** — never as a 100% overrun.

On the bundled pair: **4,638,842 AED** of priceable scope from **883 measurable elements (58%)**, reconciling as **508 priced + 375 unpriced + 643 unmeasured = 1,526**.

---

5. Mechanism C — zone map and cost link

The fallback for any model the register does not cover, and the source of the honesty metric.

`ifcZoneMap.ts` declares **14 rules**, first match wins, mapping an IFC class (with an optional storey predicate) onto Tower X's `Zone_Area` codes: footings and piles and below-ground slabs to `BASEMENT`; other slabs to `FLOORS-ALL`; columns, beams, members, walls and rebar to `STRUCTURE`; curtain walls and plates to `EXTERNAL-FACADE`; coverings to `FLOORS-B2-RF`.

Rules apply per (class, storey), never per class — and the code carries the reason, because an earlier version got it wrong:

> Testing a storey condition against the whole class is what an earlier version did, and it was badly wrong: because this model has a “Sub Level”, the below-ground slab rule fired for ALL 299 slabs and reported a flattering 100% placed.

The headline output is not the mapping but the **match rate** — how much of the model a rule set can actually place — together with `zonesWithNoGeometry`, the reverse direction:

> a structural model matches none of the MEP, finishes or landscaping budget, and saying so is the point.

`ifcCostLink.ts` then grades how firmly each element is linked, on **a third confidence scale**:

- **Direct (0.9)** — a property value on the element matches a zone code exactly. The element says where it belongs and no rule had to decide for it.
- **Grouped (0.4)** — only a class-and-storey rule placed it. True of the category, not the element.

Matching is exact after normalising case and separators, and **deliberately not a substring test**: *’* `FLOORS-ALL` *appearing inside a description like ‘concrete to floors, all levels’ is prose, not a code.”* Codes shorter than three characters are ignored as too accidental. Grouped is written first and a direct hit overwrites it — doing it the other way round would let a rule downgrade real evidence. On this model, `codeCarryingElements` is zero: the school carries no cost codes at all, which is exactly the finding.

---

6. The 4D build sequence

Once elements reach cost centres, the sheet’s progress curves can drive the model. `ifcSequence.ts` plays periods 1→12: the structure rises by each centre’s `Actual_Pct_Complete` while every element is coloured by that centre’s alert level.

What comes from the data: the pace (real S-curves, 0% at P1 rising to 66–77% by P12, per cost centre); the colour (GREEN elements appear among the AMBER around P8 because those centres genuinely recovered that month); and what never gets built (the 375 beams stay grey ghosts for the whole run, because no bill item ever paid for them — the scope gap is visible as structure that never fills in).

What is chosen is the order (assumption 8, below). Elements are ranked by storey bottom-up, then by GlobalId:

```
const STOREY_RANK = { "SUB LEVEL": 0, "01 – ENTRY LEVEL": 1, "02 – FLOOR": 2,
                      "03 – FLOOR": 3, ROOF: 4 };
// 99 for no storey, 98 for unrecognised — both sort last so they
// never displace a real level.

// GlobalId breaks ties so the sequence is identical on every run —
// a video rendered twice must not differ.
list.sort((a, b) => a.rank - b.rank || (a.globalId < b.globalId ? -1 : 1));
```

Two smaller rules, both to avoid hiding trouble: an element appears as soon as **any** of its centres reaches it (waiting for every trade would make the building lag its own concrete), and it shows the **worst** alert among its centres. Between periods, progress is interpolated but the alert is taken from the nearer period and **never blended** — there is no such thing as being 40% AMBER.

---

7. Every assumption

The eight recorded in `docs/17-ifc-boq-element-map.md`, plus six engineering assumptions that were only ever visible in source comments.

ASSUMPTION 1 — THE MODEL IS NOT THE BUILDING THE BILL IS FOR

`school_str.ifc` is a school; the bill is Tower X's. **Binding one to the other is a demo fixture.** What is real is the mechanism, and everything downstream of the BOQ item ref. The AED totals in the take-off are a demonstration that a rate library travels to an arbitrary model — not a valuation of anything. The UI says so on the page; `ifcZoneMap.ts` says so in its module header; this document says so here.

Falsified by: nothing — it is a stated fixture. Replaced by pointing the register at a real Tower X model.

ASSUMPTION 2 — REBAR IS PLACED BY STOREY, BECAUSE THE FILE CARRIES NO HOST RELATIONSHIP

A bar reinforces a specific column, slab or wall, and the bill prices rebar separately for each (2.12 raft / 2.13 columns / 2.14 slabs / 2.15 walls). **That relationship does not exist in this file** — `IFCRELASSIGNSTOPRODUCT` and `IFCRELNESTS` both have zero occurrences. Storey is the

only signal left: 560 bars on `Sub Level` → 2.12 raft, 59 bars above ground → 2.14 suspended slabs.

This is the weakest claim in the register. It is carried at **confidence 0.6**, rendered at reduced opacity in the 3D view, and stated in each row's `Basis`. Rebar to columns (2.13) and walls (2.15) is **never** assigned, because nothing in the file could justify it.

Falsified by: an export that carries the bar-to-host relationship, or a bar schedule.

ASSUMPTION 3 — FORMWORK ROWS ARE INFERRED, NOT MEASURED

A concrete element implies its formwork item — a column implies column formwork. The register says a column consumes 2.04 *and* 2.09; it does **not** measure the formwork area independently. Carried at 0.9 because the implication is a standard estimating relationship, not a guess about this model.

Falsified by: a bill that prices formwork on a basis other than the element that forms it.

ASSUMPTION 4 — 399 ELEMENTS ARE DELIBERATELY LEFT UNMAPPED

Class	Count	Why
IFCBEAM	375	**The bill prices no beam concrete** — there is no beam item in any of the 18 sections
IFCMEMBER	22	No item in this bill covers the class
IFCPLATE	2	No item in this bill covers the class

These are **reported as a scope gap, not as a mapping failure**: the model contains work the estimate never priced. Pointing them at the slab or steel item so the picture fills in would attach cost to scope the bill never carried, and inventing a number is the one thing this codebase consistently refuses to do. Coverage is capped at 74% by what Tower X's bill actually prices — not by the mechanism.

ASSUMPTION 5 — AN ELEMENT'S CONFIDENCE IS ITS WEAKEST BINDING

A slab bound at 0.9 for concrete and 0.9 for formwork reads 0.9. Had one been 0.6, the element would read 0.6. An element is only as well-placed as its shakiest link.

ASSUMPTION 6 — AN ELEMENT'S ALERT IS ITS WORST COST CENTRE

A slab whose concrete is on budget and whose formwork is drifting is painted as drifting. Averaging would hide it — the same aggregation trap the zone map's MIXED colour exists to avoid.

ASSUMPTION 7 — `9_HISTORICAL_DATA.DISCIPLINE` IS TRUNCATED IN THE SOURCE DATA

Values are cut to 18 characters (`Architectural Fini`, `Communication Syst`). **Do not use it as a join key or a display label.** Use `Package_Code` or the BOQ `Sec`. Not introduced here — it is a property of the supplied workbook, recorded because it will bite the next person.

ASSUMPTION 8 — WHICH ELEMENTS ARE BUILT, AND IN WHAT ORDER

`Actual_Pct_Complete` is per cost centre, never per element: a centre at 43% says nothing about which 43% of its 299 slabs are poured. The sequence therefore orders elements **by storey, bottom-up**, then by GlobalId for a stable tie-break, and reveals the first *n* once the centre reaches *n / total*.

That is what every 4D planning tool does and it is defensible for a concrete frame — but it is a sequence we chose. The on-screen caption says so on every frame: *"The order is assumed, the amounts are not."*

ASSUMPTION 9 — THERE IS NO UNIT CONVERSION ANYWHERE IN THE PIPELINE

Property-set numbers are summed as read and priced against m³/m² BOQ rates. There is no mm→m, no ft→m, no scaling by `IfcUnitAssignment` at any point. The pipeline **assumes the exporter wrote SI cubic and square metres**, which this Revit export does. `TakeoffPricer.Normalise` is a guard against pricing a volume at an area rate — it is not a converter and cannot rescue a model in millimetres.

Falsified by: an imperial export, or one writing mm³. The symptom would be a take-off total wrong by orders of magnitude, not an error.

ASSUMPTION 10 — UNIT RATES EXCLUDE MARGIN AND CONTINGENCY

`RateBook` derives the rate as `Direct+Indirect Amount ÷ Quantity`. The BOQ's `TOTAL Amount` — which adds margin and contingency — is deliberately not used, because the take-off is a cost question, not a price question. Comparing these figures to a contract sum will understate.

ASSUMPTION 11 — SHEET GEOMETRY IS HARDCODED

`1_BOQ` header at row 5, data from row 6; `9_HISTORICAL_DATA` header at row 5, data from row 6; `2_ESTIMATE_NORMS`, `3_BOQ_MAPPING`, `4_ESTIMATE_DATASHEET` header at row 4. Periods are constrained to 1–12, and only rows whose `Package_Code` starts with `EP-` are kept — which drops the workbook's trailing `AC_Cumul` block. Header names are matched after lower-casing and stripping *all* whitespace, because several carry embedded newlines (`"Qty/\nUnit Work"`).

Falsified by: a re-issued workbook with a row inserted above the header.

ASSUMPTION 12 — `AC_AED_PERIOD` IS CUMULATIVE DESPITE ITS NAME

Read into `AcCumulative` with the comment *"cumulative in this workbook"*. A genuine naming trap in the supplied data; treating it as a per-period increment would double-count actual cost.

ASSUMPTION 13 — A ZONE'S CPI IS NOT PAINTED BELOW 1% OF BUDGET SPENT

`MaterialityFloor = 0.01` in `ModelController`. Under 1% of BAC booked as actual cost, a CPI is statistically meaningless, and the zone reports `INSUFFICIENT_COST` rather than a flattering or alarming colour.

ASSUMPTION 14 — THE REGISTER IS NOT REGENERATED BY ANY BUILD STEP

`generate_map.py` is run by hand. If `school_str.ifc` is ever replaced, the register does not follow it. **Drift is detected, not prevented**: `buildElementIndex` counts register rows the loaded model does not contain as `notInModel` and surfaces it in the UI, and `IfcElementMapTests` pins 1,526 / 1,127 / 399 so the build fails if the committed CSV stops matching its own claims.

---

8. Where the three mechanisms disagree

No single existing document shows this, and each of these is a live trap for the next reader.

Three confidence scales that look alike. The register uses `0.9 / 0.6 / 0.0`; the cost-link tiers use `Direct 0.9 / Grouped 0.4`; paint opacity uses `1 / 0.55` for the register and `1 / 0.7 / 0.3` for the build sequence. `paintIfcByCostCentre` keys opacity off the **register's** confidence; `paintIfcByCost` keys it off the **link tier**. A `0.9` in one place does not mean the same thing as a `0.9` in another.

Two class→item tables that disagree. The register maps eight BOQ items (2.04, 2.05, 2.06, 2.09, 2.10, 2.11, 2.12, 2.14). `TakeoffRateMap` prices four (2.04, 2.05, 2.06, 2.11). So column and wall formwork (2.09, 2.10) and all rebar (2.12, 2.14) **reach cost centres and drive the 4D sequence but are never priced in the take-off**. Both tables are defensible for their own question — the register asks what an element consumes, the rate map asks what can be measured off geometry — but they are not the same table and should not be quoted as one.

`IFCWALLSTANDARDCASE` measures but cannot be priced. It is in `CONSIDERED`, in `MEASURED_CLASSES`, in `ZONE_RULES` and in `generate_map.py`'s `rules()` — but `TakeoffRateMap` has a rule only for `IFCWALL`. Harmless on this model (6 `IFCWALL`, zero standard-case) and a silent trap on the next one.

`BoqSec` is generated and never consumed. The column exists in the CSV and is written for every mapped row, but neither the loader nor the API reads it. BOQ sections play no part in the join.

---

9. Verification and regeneration

```
# deterministic — run it twice and the diff is empty
python3 tools/ifc_boq_map/generate_map.py
```

It prints coverage, rows per BOQ item, and the scope gap. **Hand-editing the register is expected and supported.** A QS who disagrees that a column's formwork belongs to 2.09 can change that row; the join-integrity tests will fail the build if an edit points at a BOQ item that does not exist or reaches no cost centre.

Three test suites hold the connection in place, all reading the committed artefacts rather than fixtures:

Suite	Facts	Asserts
`IfcElementMapTests`	11	every mapped item exists in `1_BOQ`; every item resolves to a cost centre through `WBS_Code`; item refs and WBS codes are the same set, one centre per item; coverage is 1,127 / 1,526 and mapped + unmapped accounts for every element; beams are reported as scope the bill never priced; rebar is the only thing below 0.9 and says why; every rule ships a non-empty rationale; commas inside a quoted rationale survive the parser
`TakeoffPricingTests`	15	the `priced + unpriced + unmeasured == total` tie-out, and the unit-agreement guard
`DataContractTests`	—	the workbook is still 2,076 rows / 173 cost centres

Known limits. Coverage is capped at 74% by what Tower X's bill prices. The register binds one specific model to one specific project — loading a different IFC through the file picker leaves it unresolved and the tab falls back to zone-level placement (Mechanism C). And the whole first arrow remains what it has been from the start: a declared judgement, shipped in a file a QS can read and argue with.